

Short Questions 8

Josh Manchester
Department of Computer Science
University of Colorado Colorado Springs
Colorado Springs, CO 80918
jmanches@uccs.edu

Abstract

This paper addresses three topics at the intersection of deep learning and reinforcement learning: the softmax function and its role in policy networks, convolutional filters and their benefits in neural networks, and the potential use of recurrent neural networks in RL for handling temporal dependencies and partial observability.

Question 1

Give the formula for the softmax function. Where is it used in a neural network? What does it accomplish? In a Reinforcement Learning setup using neural networks, what benefits does the use of softmax bring?

The softmax function takes a vector of raw scores (logits) $\mathbf{z} = [z_1, z_2, \dots, z_K]$ and converts them into a probability distribution:

$$\text{softmax}(z_i) = \frac{e^{z_i}}{\sum_{j=1}^K e^{z_j}}, \quad i = 1, \dots, K \quad (1)$$

Each output is positive and between 0 and 1, and all outputs sum to exactly 1. The exponential amplifies differences between logits—large values get pushed larger, small values get pushed smaller—so the highest-scoring option stands out clearly.

In a standard neural network, softmax is placed at the output layer for classification tasks. The network's final hidden layer produces raw logits, and softmax converts them into class probabilities. We then pick the class with the highest probability, or use the full distribution to compute a cross-entropy loss for training.

In reinforcement learning, softmax serves a different but equally important role: it turns a policy network into a stochastic policy. Instead of outputting class labels, the network outputs action probabilities $\pi(a | s)$ for each action a given state s . The agent then samples from this distribution rather than always picking the greedy action. This provides natural exploration—actions with lower estimated value still get selected occasionally, but in proportion to how good they look relative to the alternatives (Sutton and Barto 2018). Unlike ϵ -greedy, which explores uniformly at random, softmax exploration is structured: near-optimal actions

get tried more often than clearly bad ones. This makes policy gradient methods like REINFORCE possible, since the gradient $\nabla_{\theta} \log \pi(a | s; \theta)$ is well-defined and differentiable through the softmax output (Sutton & Barto, Ch. 13).

Question 2

What is a convolution or filter as used in an artificial neural network (ANN)? What are benefits of using convolutions in an ANN?

A convolution (or filter) is a small matrix of learnable weights—typically 3×3 or 5×5 —that slides across an input (such as an image) and computes a weighted sum at each position. At every location, the filter performs an element-wise multiplication (Hadamard product) with the corresponding patch of the input, then sums the results into a single value. Doing this across the entire input produces an *activation map* that highlights where a particular feature—like an edge, a corner, or a texture—appears in the input. Multiple filters run in parallel, each learning to detect a different feature, so one convolutional layer produces a stack of activation maps (Goodfellow et al. 2016).

Convolutions bring several key benefits over fully connected (dense) layers. First, **parameter sharing**: the same small filter is reused across every spatial position, so a 3×3 filter has only 9 weights regardless of whether the input is 28×28 or 256×256 . A dense layer connecting every pixel to every neuron would need millions of parameters for the same input. Second, **translation invariance**: because the filter slides everywhere, it detects a feature no matter where it appears in the image. An edge in the top-left corner activates the same filter as the same edge in the bottom-right. Third, **hierarchical feature learning**: stacking multiple convolutional layers lets the network build up from low-level features (edges, lines) in early layers to high-level features (shapes, objects) in deeper layers. This is why convolutional neural networks are the standard approach for processing visual input in deep RL—DQN used a CNN to go directly from raw Atari pixels to action values (Mnih et al. 2015).

Question 3

Can recurrent neural networks (RNNs) be potentially used in Reinforcement Learning? In what way?

Standard RL assumes the agent can fully observe the current state, but many real problems violate this. A robot with a single camera cannot see behind itself. An agent in a card game does not see the other players' hands. When the observation does not capture the full state, we have a partially observable environment, and a single frame is not enough for the agent to make a good decision—it needs memory of what it has seen before.

RNNs address this by maintaining a hidden state that carries information from previous time steps. At each step, the network takes in the current observation *and* its own hidden state from the previous step, producing both an output and an updated hidden state. This hidden state acts as the agent's memory, letting it integrate information across multiple observations to build a more complete picture of the true state.

Hausknecht and Stone (2015) demonstrated this directly by replacing the first fully connected layer in DQN with an LSTM (a type of RNN that handles long-term dependencies better than vanilla RNNs). Their Deep Recurrent Q-Network (DRQN) was tested on Atari games where they deliberately removed portions of the screen, making the game partially observable. DRQN outperformed standard DQN in these conditions because the LSTM could remember information from previous frames that was no longer visible. So where a CNN-based DQN needs the full state in the current frame, an RNN-based architecture can piece together what it needs from a sequence of incomplete observations. Think of it like exploring a dark room with a flashlight—at any moment, we only see a small cone of light, but as we sweep the beam around, we build a mental map of the whole room: where the furniture is, where the walls are, what we have already passed. The LSTM's hidden state is that mental map. Each new observation adds to it, and the agent makes decisions based on the accumulated picture rather than just what the flashlight is pointing at right now.

AI Use Statement

Claude (Anthropic) was used to format my answers in AAAI LaTeX format.

References

- Goodfellow, I.; Bengio, Y.; and Courville, A. 2016. *Deep Learning*. Cambridge, MA: MIT Press. Available free at <http://www.deeplearningbook.org>.
- Hausknecht, M.; and Stone, P. 2015. Deep Recurrent Q-Learning for Partially Observable MDPs. In *AAAI Fall Symposium on Sequential Decision Making for Intelligent Agents (AAAI-SDMIA)*.
- Mnih, V.; Kavukcuoglu, K.; Silver, D.; Rusu, A. A.; Veness, J.; Bellemare, M. G.; Graves, A.; Riedmiller, M.; Fidjeland, A. K.; Ostrovski, G.; et al. 2015. Human-level control through deep reinforcement learning. *Nature*, 518(7540): 529–533.
- Sutton, R. S.; and Barto, A. G. 2018. *Reinforcement Learning: An Introduction*. Cambridge, MA: MIT Press, 2nd edition. Available free at <http://incompleteideas.net/book/the-book-2nd.html>.